

Manuale Tecnico

CINEMAX - MANUALE TECNICO

Sistema di Gestione e Prenotazione Spettacoli Cinematografici

FRONTESPIZIO

- **Titolo del Documento:** Manuale Técnico - Architettura e Implementazione CineMax
- **Autori:**
 - Matteo Luraghi (Matricola: 765632 - Sede: Varese)
 - Fabio Maffiolini (Matricola: 765567 - Sede: Varese)
- **Data di Rilascio:** 21 Giugno 2026
- **Versione Documento:** v1.0
- **Destinatari:** Docenti Universitari di Informatica, Sviluppatori, Personale tecnico addetto alla manutenzione.

INDICE

1. STRUTTURA DELL'APPLICAZIONE
2. LE SCELTE ARCHITETTURALI
3. LE STRUTTURE DATI UTILIZZATE
4. LE SCELTE ALGORITMICHE
 - 4.1 Sicurezza delle Password
 - 4.2 Lettura avanzata dal file CSV e Parsing testuale
 - 4.3 Algoritmo di ricerca nell'HashMap
 - 4.4 Ordinamento con .sort() e interfaccia Comparable
5. IL FORMATO DEI FILE E LA LORO GESTIONE
6. L'USO DI PATTERN (DESIGN PATTERN BASE)
7. DOCUMENTAZIONE JAVADOC

Questo documento descrive come è stato programmato e strutturato il progetto **CineMax**, un sistema per piccoli cinema monosala da 200 posti.

1. STRUTTURA DELL'APPLICAZIONE

Il codice del progetto è stato organizzato in *package* distinti, raggruppando le classi in base al loro compito. All'interno della directory `src/cinemax/` troviamo:

- **modelli** : Contiene le classi che definiscono la struttura dei dati dell'applicazione (le entità, come `Utente` , `Cliente` , `Prenotazione`). In questo modulo è stato fatto un uso ponderato dell'ereditarietà: è stata creata una classe base astratta `Utente` che raggruppa tutti i dati e i comportamenti comuni (es. credenziali, anagrafica). Da questa derivano implementazioni specifiche come `Cliente` , `Bigliettaio` e `Proiezionista` . Questa scelta architetturale garantisce un'ottima estensibilità: se in futuro fosse necessario aggiungere nuovi attributi o metodi specifici solo per un determinato tipo di utente, basterà modificare la sua classe dedicata, senza intaccare il resto del codice o appesantire inutilmente la struttura delle classi.
- **gestori** : Contiene le classi incaricate di gestire la logica e l'accesso ai dati (come `GestoreUtenti` e `GestorePrenotazioni`). In queste classi vengono implementati i controlli, le regole di validazione e i metodi per il salvataggio o la lettura dai file.
- **menu** : Costituisce l'interfaccia utente testuale (TUI). I menu sono differenziati in base al ruolo dell'utente che ha effettuato l'accesso (`MenuCliente` , `MenuBigliettaio` , ecc.), garantendo che ognuno abbia accesso solo alle funzionalità di propria competenza.
- **eccezioni** : Ospita classi *custom* che estendono le eccezioni standard di Java (come `UtenteUsernameException`), permettendo di identificare e gestire gli errori in modo specifico e controllato, separandoli dagli errori generici di sistema.
- **Cinemax.java** : Rappresenta l' *Entry Point* contenente il metodo `main` . Il suo scopo è avviare l'applicazione, istanziare i vari gestori, caricare i dati iniziali dai file e invocare il menu interattivo.

2. LE SCELTE ARCHITETTURALI

- **Divisione a livelli:** Il codice è diviso in tre parti logiche: la grafica (i menu a riga di comando), la logica (i gestori) e i dati (il salvataggio su file).
- **Gestione in memoria (RAM) per essere più veloci:** Leggere e scrivere continuamente su file rallenterebbe molto il programma. Per questo motivo, quando il programma parte, legge tutti i dati dai file (CSV) e li salva in variabili nella RAM (come liste e mappe). Da quel momento in poi, tutte le modifiche vengono fatte solo in RAM in modo istantaneo. I file vengono aggiornati solo alla fine, quando l'utente sceglie di uscire dal programma.
- **Gestione dei ruoli:** A seconda dell'utente che effettua il login, il programma mostra opzioni diverse, limitando l'accesso in base ai permessi (Cliente, Bigliettaio o Proiezionista).

3. LE STRUTTURE DATI UTILIZZATE

Per memorizzare i dati in RAM e renderli facili da cercare, abbiamo usato in modo combinato le collezioni di Java:

- **HashMap**: È la struttura dati principale ed è stata impiegata **in tutti e tre i gestori** (`GestoreUtenti`, `GestorePrenotazioni`, `GestoreProiezioni`). Abbiamo scelto le mappe perché associano una chiave univoca (ad esempio lo username dell'utente, il codice testuale della prenotazione o la data e l'ora della proiezione) direttamente all'oggetto corrispondente. Questo ci permette di trovare o verificare un dato in modo immediato, senza dover scorrere tutta la lista riga per riga, rendendo il programma estremamente veloce.
- **ArrayList**: Abbiamo usato le liste principalmente come strutture di supporto temporanee quando dovevamo filtrare, ordinare o impaginare i dati. Ad esempio, quando l'utente cerca "tutti i film di oggi" o "tutte le sue prenotazioni", i risultati trovati all'interno della `HashMap` vengono prelevati, inseriti in un `ArrayList` e mostrati a schermo in ordine.

4. LE SCELTE ALGORITMICHE

4.1 Sicurezza delle Password

Per evitare di salvare le password in chiaro, abbiamo usato un algoritmo di crittografia chiamato **SHA-256**.

Quando un utente si registra, la sua password viene trasformata in un hash. Anche nei file salviamo solo l'hash. Quando l'utente effettua il login,

trasformiamo la password che ha inserito in hash e controlliamo se è uguale a quello salvato.

4.2 Lettura avanzata dal file CSV e Parsing testuale

La lettura dei dati avviene ciclando ogni singola riga del file testuale. In casi semplici, usiamo il metodo standard `.split(",")` di Java per separare i dati.

Tuttavia, per i file più complessi, abbiamo implementato un algoritmo di parsing personalizzato denominato `analizzaRigaCSV`. Questo algoritmo scorre la riga carattere per carattere ed è in grado di capire se una virgola fa parte del testo oppure se è un vero separatore. Lo fa controllando, tramite un flag booleano, se in quel momento il testo si trova dentro alle virgolette.

4.3 Algoritmo di ricerca nell'HashMap

A livello algoritmico, la scelta dell' `HashMap` si basa sul concetto di "tabella hash". Quando inseriamo un elemento (ad esempio un utente), l'algoritmo calcola una funzione matematica sulla chiave (lo username) per ottenere un numero intero (l' *hash code*). Questo numero indica la "scatola" (*bucket*) di memoria esatta in cui andare a salvare o recuperare i dati. Questo algoritmo permette di evitare di fare ricerche sequenziali lente con i cicli `for`: l'accesso al dato è diretto e richiede un tempo costante, indicato matematicamente come $O(1)$.

4.4 Ordinamento con `.sort()` e interfaccia Comparable

Quando abbiamo la necessità di mostrare a video liste ordinate di proiezioni o prenotazioni (memorizzate negli `ArrayList`), usiamo il metodo `.sort(null)` fornito da Java.

Per l'ordinamento di liste di oggetti, Java utilizza l'algoritmo **TimSort** (un algoritmo di ordinamento ibrido e stabile derivato da **Merge Sort** e **Insertion Sort**). La complessità computazionale di questo algoritmo è:

- **Complessità temporale:**
 - Caso peggiore (Worst-case): $O(N \log N)$
 - Caso medio (Average-case): $O(N \log N)$
 - Caso migliore (Best-case): $O(N)$ (quando la lista è già parzialmente o totalmente ordinata)
- **Complessità spaziale:** $O(N)$ spazio di memoria ausiliario richiesto per la fusione degli array.

Le nostre classi (ad esempio `Proiezione`) sono state progettate per implementare l'interfaccia `Comparable`. Grazie a questa interfaccia, abbiamo sovrascritto il metodo `compareTo()`, istruendo il sistema a confrontare direttamente la data e l'ora (`dataOra`): in questo modo le proiezioni vengono ordinate in automatico in senso cronologico crescente.

5. IL FORMATO DEI FILE E LA LORO GESTIONE

Per salvare i dati in modo permanente abbiamo scelto il formato **CSV con i valori separati da virgola**.

- **Gestione dei percorsi:** I file si trovano nella cartella `data/`.
- **Lettura e scrittura su file:** Usiamo classi standard di Java come `FileReader` e `BufferedReader` per leggere i file riga per riga in modo efficiente. Per salvare, usiamo `FileWriter` e `PrintWriter` per scrivere tutti i dati aggiornati sul file prima che il programma si chiuda.

6. L'USO DI PATTERN (DESIGN PATTERN BASE)

Abbiamo usato alcune buone pratiche tipiche della programmazione a oggetti:

- **Passaggio dei riferimenti:** Invece di far creare i gestori dentro ogni menu o usare variabili globali (che creano confusione), li creiamo una volta sola nel `main` e li passiamo "come parametro" ai costruttori dei menu che ne hanno bisogno:Java

```
// Creiamo il menu passando i gestori già pronti
new MenuCliente(gestoreProiezioni, gestorePrenotazioni,
utente);
```

- **Polimorfismo:** Quando leggiamo gli utenti dal file, in base a una parola scritta nel file (es. "CLIENTE" o "BIGLIETTAIO"), il nostro codice decide tramite un `if` quale tipo specifico di oggetto creare e lo inserisce nella mappa usando il tipo generale `Utente`.

7. DOCUMENTAZIONE JAVADOC

Il codice è stato interamente commentato seguendo lo standard **JavaDoc**.

Davanti a ogni classe e metodo importante c'è un commento speciale `/** ... */` che descrive a cosa serve il metodo, cosa riceve in ingresso (`@param`) e cosa

restituisce in uscita (`@return`). Questo è fondamentale per poter generare automaticamente un manuale del codice HTML leggibile per eventuali altri studenti o programmatori che volessero aggiungere funzioni al nostro progetto.